

BOA HYDRAULICS R&D DEPARTMENT

Smart Die Validation for QAQC System

Final Report

Haoyu Pang

2026-02-20



Table of Contents

1. Introduction	1
2. Project Overview and Objectives	1
3. Approach Exploration	2
3.1. Functional Requirements for Automated Die Validation	2
3.2. Candidate Technical Approaches Considered	2
3.3. Prototype Candidate Selection	3
4. Data Matrix Based Die Identification	4
4.1. Prototype Implementation	5
4.1.1. Code Generation and Decoding Mechanism	5
4.1.2. Preprocessing Techniques	6
4.1.3. Live Validation Framework	8
4.2. Experimental Setup	9
4.2.1. Imaging Hardware	9
4.2.2. Camera Mounting	9
4.2.3. Camera Configuration	10
4.2.4. Die and Symbol Configuration	11
4.2.5. Lighting and Environmental Conditions	12
4.2.6. Validation Parameters	12
4.3. Performance Evaluation	12
4.3.1. Fixed-window decoding reliability	12
4.3.2. Live timing mode	13
4.4. Observed Limitations	14
4.4.1. Camera Mounting Constraints	14
4.4.2. Industrial Marking Constraints	15
4.4.3. Additional Manufacturing Cost and Preparation Time	15
4.4.4. Surface Contamination Sensitivity	15
4.4.5. QAQC Placement Model Interaction	15
5. Machine Learning Based Die Verification	16
5.1. Data Collection	16
5.1.1. Capture Setup and ROI Normalisation	16
5.1.2. Controlled variation protocol	17
5.1.3. Dataset Splitting and Closed/Open-Set Configuration	19
5.1.4. Dataset Structure	20
5.2. Model and Training	21
5.2.1. Model Choice	21
5.2.2. Loss Function	22
5.2.3. Data Augmentation	24
5.2.4. Training Strategies	24
5.2.5. Validation and Checkpoint Selection	25
5.2.6. Validation Performance (Best Training Run)	26
5.3. Performance Evaluation	29
5.3.1. Live Test Setup	29
5.3.2. Decision Logic	30

5.3.3. Results	31
6. QAQC Integration Design and Implementation	33
6.1. Approach Selection	33
6.2. Integration Point	34
6.3. Database Changes	35
6.4. Step 3 Flag Changes	36
6.5. Die Validation Workflow	37
6.5.1. High Level Overview	37
6.5.2. Different Cases and Backend Logic	38
6.5.3. Skip Handling Change	41

List of Figures

Figure 1 Available Die Sets	5
Figure 2 6×6 mm, payload 50	5
Figure 3 Camera	9
Figure 4 Camera Mounting Position	10
Figure 5 Camera View	11
Figure 6 DataMatrix Attached to the Die	11
Figure 7 Time to Reach Three Matches	14
Figure 8 Example Cropped ROI - Die 17	16
Figure 9 Baseline Variations	17
Figure 10 Angle Variations	18
Figure 11 Lighting Variations	18
Figure 12 EfficientNet-B0 with Classifier Head	21
Figure 13 ArcFace Loss	22
Figure 14 Softmax vs ArcFace Embedding Space	23
Figure 15 Train Loss	29
Figure 16 Checkpoint TAR@FAR 0.01	29
Figure 17 Close Set Test Results	32
Figure 18 Open Set Test Results	33

List of Tables

Table 1 Performance Results per Attempt	13
Table 2 Training/Validation/Test split	20
Table 3 Open-set Die Collection	20
Table 4 Data Augmentations	24
Table 5 Training Configuration	27
Table 6 TAR@FAR for Hard Negatives	28
Table 7 TAR@FAR for All Negatives	28
Table 8 Columns Added to the Database	35
Table 9 Step 3 Backend PKG	36
Table 10 Step 3 Frontend PKG	37

1. Introduction

Hydraulic hose assembly is a precision-dependent manufacturing process in which the crimping operation plays a critical role in determining joint integrity, pressure containment capability, and long-term operational reliability. Within BOA's production workflow, the Quality Assurance and Quality Control (QAQC) system is deployed to provide operator guidance and automated validation across multiple stages of hose assembly. These validation stages aim to minimise assembly errors and ensure compliance with predefined crimp specifications prior to and after the crimping process.

Despite automated verification mechanisms in the existing QAQC framework, selecting and installing the crimping die set prior to crimp initiation remains a manual process. Each crimping machine uses multiple interchangeable die sets of varying sizes, and operators must select the appropriate die from crimp specification charts. Incorrect die selection may result in under- or over-crimping, potentially leading to joint failure, leakage, reduced service life, or safety risks during operation. As this step is currently outside the automated verification loop, it represents a critical source of potential assembly error within the placement stage of the QAQC workflow.

To address this limitation, the Smart Die Set Validation project was initiated to investigate the feasibility of implementing an automated die verification mechanism that detects and validates the installed die set prior to crimping. By integrating die validation into the QAQC workflow at the placement stage, the project aims to reduce the risk of incorrect die selection entering the production process while maintaining compatibility with existing system architecture and real-world workshop operating conditions.

2. Project Overview and Objectives

The Smart Die Set Validation project aims to design and prototype an automated system capable of identifying the die installed in the crimper and verifying its compliance with the required die specified for a given hose-fitting assembly. The intended functionality of the system includes identifying installed die size, comparing it against the required die set defined in existing crimp specification data sources, and providing operator-facing feedback to warn or inhibit progression to the crimping stage when a non-compliant die selection is detected.

To ensure compatibility with existing production workflows, the validation mechanism was designed to operate within Step 3 - Placement Check of the QAQC process. The validation is therefore triggered when the operator attempts to proceed to the crimping stage for an assembly whose required die set has not yet been verified in the current branch context. Upon operator-initiated validation, the system compares the physically installed die to the required die and produces a PASS, FAIL, or UNCERTAIN outcome, which determines whether the crimping process may proceed.

Multiple technical approaches for die set validation were investigated during the project, including deterministic identification using laser-marked DataMatrix codes on die segments and machine learning-based visual verification using embedding similarity derived from captured die images. These approaches were evaluated based on accuracy, environmental robustness, implementation complexity, and integration feasibility within the QAQC system.

The primary objective of this internship project was to develop a functional prototype of the selected validation approach and demonstrate its integration with the existing QAQC workflow under typical production-room conditions. Additional objectives included evaluating the die validation system's detection accuracy and response time, documenting its architecture and implementation logic, and providing recommendations for future production deployment.

3. Approach Exploration

To address the limitation of manual die selection in the current QAQC workflow, the Smart Die Set Validation project investigated multiple approaches to automatically detect and verify the die installed in the crimper prior to crimp initiation. Candidate detection technologies were required to operate reliably under real workshop conditions, including oil contamination, variable lighting, reflective metallic surfaces, and operator interaction.

3.1. Functional Requirements for Automated Die Validation

Based on observation of live crimping operations and review of the existing QAQC workflow, the automated die validation system was required to satisfy the following operational and environmental constraints:

- Perform non-contact die identification in place
- Operate reliably under workshop contamination
- Tolerate variable lighting and reflective metallic surfaces
- Require minimal modification to existing die sets or machine hardware
- Provide validation results without significant disruption to production workflow
- Integrate with QAQC logic to support branch-level verification
- Minimise operator intervention during validation

3.2. Candidate Technical Approaches Considered

Following initial system analysis, multiple technical solution pathways were explored for die identification and verification. These approaches were broadly categorised into three groups: explicit identification methods, geometry-driven measurement approaches, and vision-based recognition methods.

Explicit Identification Methods

Explicit identification approaches involved modifying the die set to carry machine-readable identifiers that the QAQC system could directly decode. Examples considered included:

- Laser-marked DataMatrix codes on die segments
- Human-readable engraved identifiers with optical character recognition (OCR)
- Colour-coded markings are distinguishable via hue-based classification

These methods provide deterministic identification by directly mapping detected identifiers to stored die specifications. However, their effectiveness depends on the durability of applied markings and resistance to contamination, occlusion, and specular reflection.

Geometry-Driven Measurement Approaches

Geometry-driven approaches aimed to infer die identity through measurable physical characteristics such as inner diameter, tooth spacing, or segment profile. Investigated concepts included:

- Keypoint tracking-based geometric measurement of die features
- Inner diameter circle fitting

Adjacent die sizes differ by only 4 mm in closed inner diameter, requiring measurement accuracy within tight tolerances to avoid incorrect acceptance of visually similar dies. Achieving this level of repeatability would require highly controlled camera alignment and consistent die positioning during validation.

Such approaches would be sensitive to pose variability introduced by manual operator setup, increasing calibration complexity and reducing robustness under uncontrolled production conditions.

Vision-Based Recognition Methods

Vision-based approaches relied on machine vision techniques to identify die sets based on visual appearance rather than explicit identifiers or geometric measurements. These included:

- CNN-based multi-class classification
- Verification-style visual matching using learned embeddings

Learning-based verification methods encode die images into feature embeddings and compare them against reference embeddings of known die sets, enabling identity confirmation through similarity-based matching rather than direct classification.

3.3. Prototype Candidate Selection

The candidate approaches were evaluated qualitatively for suitability for deployment under realistic workshop operating conditions. In particular, approaches were assessed against the following practical implementation criteria:

- Environmental robustness under oil contamination and metallic surface reflectivity
- Integration feasibility within existing crimper and camera configurations
- Implementation complexity at the prototype stage
- Repeatability and reliability of die verification
- Hardware and calibration requirements
- Scalability to support the full range of production die sets

Explicit identification methods based on colour coding were excluded due to sensitivity to lighting variations and surface contamination. Similarly, human-readable engraving combined with OCR introduces segmentation challenges on oil-stained, worn or reflective surfaces, increasing the risk of false reads.

Geometry-driven measurement approaches were also excluded from prototype development due to their dependence on tightly controlled pose and calibration stability in order to achieve the measurement accuracy required to differentiate adjacent die sizes.

Conventional multi-class CNN-based classification requires the model to distinguish between all possible die types simultaneously when adjacent die sizes are visually similar. Furthermore,

supporting new die sizes through classification would require retraining on expanded labelled datasets, increasing system maintenance complexity.

Following this feasibility filtering process, two candidate approaches were selected for prototype implementation and empirical evaluation:

1. Deterministic die identification using **laser-marked DataMatrix codes**
2. **Machine learning-based die verification** using visual embedding similarity

The DataMatrix-based identification approach offers several practical advantages for QAQC integration. As a deterministic decoding method, it provides unambiguous die identity mapping with minimal susceptibility to visual similarity between adjacent die sizes. The decoding process is computationally lightweight and orientation independent, supporting validation within operator workflow cycle-time constraints. Furthermore, once applied, the DataMatrix markings allow direct traceability between physical die sets and stored specification records, simplifying system logic and reducing the risk of incorrect acceptance due to appearance ambiguity. These approaches are well-suited to applications that require high verification certainty and predictable runtime performance.

In contrast, the embedding-based visual verification approach enables non-invasive die identification by leveraging learned feature representations derived from captured die images. By performing similarity-based matching rather than multi-class classification, the system can verify installed die identity without requiring retraining when new die sizes are introduced, provided suitable reference embeddings are available. This improves scalability and reduces long-term maintenance overhead as production die inventories expand. Additionally, the absence of required physical modification to existing die sets supports retrofit deployment within current workshop environments, allowing validation capability to be introduced without interrupting established tooling workflows.

Together, these approaches represent complementary implementation strategies: one prioritising deterministic identification reliability through explicit marking, and the other prioritising deployment flexibility and scalability through learned visual verification. Their selection for prototype development, therefore, enables comparative evaluation of marking-based and perception-based validation pathways with respect to accuracy, robustness, integration complexity, and response time within the QAQC workflow.

4. Data Matrix Based Die Identification

To evaluate the feasibility of DataMatrix-based die identification, a prototype implementation using DataMatrix-coded dies was developed and experimentally assessed under typical production-room conditions. The prototype was implemented as a standalone validation mechanism to assess the reliability of DataMatrix-based die recognition. The following subsections describe the prototype implementation, the experimental setup, the observed performance characteristics, and the limitations identified during the empirical evaluation.

4.1. Prototype Implementation

The Data Matrix-based die identification system encodes die identity as a machine-readable Data Matrix symbol physically attached to each die segment, and verifies the installed die by decoding the symbol in real time using a fixed camera.

For experimental evaluation, Data Matrix labels were generated in black-and-white, printed on paper, and attached to all eight die segments using transparent tape. The evaluated die size range consisted of:

17, 20, 24, 28, 32, 36, 40, 44, 50



Figure 1: Available Die Sets

Only the 50-size die was tested in a 6 mm physical code size, as this die presents the most challenging decoding condition. The 50 die has the largest inner diameter and therefore the smallest available frontal die face area for label attachment. Consequently, the 6 mm Data Matrix symbol represents the smallest feasible code size and the most demanding optical scenario. If reliable decoding is achieved for this configuration, decoding of larger die sizes with larger available marking area is expected to be comparatively more straightforward.

The prototype implementation consisted of three main components:

- Data Matrix code generation and decoding mechanism
- Image preprocessing pipeline
- Real-time live validation framework

4.1.1. Code Generation and Decoding Mechanism

Data Matrix Generation

Data Matrix (ECC200) symbols were generated using `pylibdmtx`.



Figure 2: 6 × 6mm, payload 50

The generator provides control over:

- Payload string
- Encoding format (default: UTF-8)
- Physical output size in millimetres
- DPI scaling (default: 600)
- Foreground and background colour (default: black and white)
- Optional PDF export for print accuracy

ECC200 was selected because:

- It provides built-in Reed-Solomon error correction.
- It supports compact encoding for short payloads.
- It enables deterministic one-to-one mapping between payload and die size.

Data Matrix Decoding

Decoding was implemented using `pylibdmtx`.

The `pylibdmtx` decoder:

- Accepts an image input
- Searches for ECC200 symbols
- Returns decoded byte payload(s)
- Converts them to UTF-8 strings
- Returns none if no symbol is detected

`pylibdmtx` was selected over alternative barcode libraries, such as OpenCV's built-in barcode module, or general-purpose multi-format scanners (e.g., ZXing-based wrappers), for several technical reasons. First, `libdmtx` is optimised explicitly for ECC200 Data Matrix decoding and provides mature, production-tested performance for small, high-density symbols. General-purpose scanning libraries often prioritise multi-format flexibility (QR, EAN, Code128, etc.) rather than optimised performance for small industrial Data Matrix codes. Second, it allows explicit control over decoding parameters, including symbol count limits and timeout behaviour, which is critical for enforcing deterministic cycle-time constraints within the QAQC workflow. Third, the library operates reliably on raw grayscale image buffers without requiring complex image format conversions, reducing preprocessing overhead.

4.1.2. Preprocessing Techniques

To improve robustness to decoding under workshop conditions, several optional preprocessing techniques were implemented in the live decoder framework.

These techniques were configurable and evaluated experimentally.

Grayscale Conversion

All captured ROI images were converted from BGR to grayscale prior to decoding. The `pylibdmtx` decoder operates on luminance intensity rather than colour information. Grayscale conversion reduces computational complexity and removes colour-dependent variation introduced by lighting conditions or surface reflections. This step serves as the baseline preprocessing stage for all subsequent enhancement operations.

Region of Interest (ROI) Restriction

The system supports:

- Fixed pixel ROI
- ArUco-marker-based ROI detection with perspective correction
- Secondary inner crop after correction

ROI restriction reduces background clutter and constrains the search space for the decoder, thereby:

- Reducing decode time
- Improving robustness against false detections
- Improving contrast consistency

The application of perspective correction provides:

- Geometric normalisation: Restores a fronto-parallel view of the die face.
- Module consistency: Preserves square module structure required for reliable ECC200 decoding.
- Improved robustness: Reduces false negatives caused by perspective distortion.

Upscaling

ROI upscaling via bicubic interpolation was implemented to increase effective module pixel density before decoding.

This was beneficial when:

- The symbol occupied a small pixel footprint
- Camera mounting distance could not be reduced

Contrast Enhancement (CLAHE) (optional, off by default)

Contrast Limited Adaptive Histogram Equalisation (CLAHE) was optionally applied prior to decoding.

Rationale:

- Metallic surfaces produce local illumination variations.
- Oil contamination reduces global contrast.
- CLAHE enhances local module boundaries without globally amplifying noise.

Adaptive Thresholding (optional, off by default)

Adaptive Gaussian thresholding was optionally applied to convert grayscale images to binary.

This can improve decoding when:

- Background illumination varies spatially
- Surface reflections introduce gradients

However, excessive thresholding may:

- Remove fine module detail
- Increase aliasing for tiny symbols

4.1.3. Live Validation Framework

To evaluate the feasibility of Data Matrix-based die identification under realistic production conditions, a real-time validation framework was developed using OpenCV and the pylibdmtx decoding library.

The framework simulates the intended die validation behaviour by continuously capturing images from a fixed camera, extracting a defined region corresponding to the die face, and attempting Data Matrix decoding at controlled time intervals.

Real-Time Capture and ROI Strategy

The system negotiates the highest available camera resolution to maximise symbol pixel density while maintaining stable frame acquisition.

To reduce computational load and minimise false detections, decoding is restricted to a Region of Interest (ROI). For the primary evaluation of the 6 mm symbol on the 50 die, a ArUco-based ROI was used to represent a stable, production-style mounting configuration.

Hard Timeout Protection

To guarantee responsiveness, decoding is executed with an enforced timeout mechanism. If a decode attempt exceeds the specified time threshold, the attempt is aborted and classified as “No Read”.

This design ensures that occasional difficult frames do not stall the system, preserving operator workflow continuity — an essential requirement for QAQC integration.

Multi-Cycle Validation Logic

Rather than relying on a single successful decode, the framework implements a multi-cycle validation strategy.

A validation attempt consists of:

- A fixed number of decode cycles
- A minimum required number of correct matches
- If the required number of matches is achieved within the attempt window, the result is classified as PASS; otherwise, FAIL.

This approach improves robustness against:

- Temporary reflections
- Partial occlusion
- Motion blur
- Lighting fluctuations

The strategy reflects realistic production behaviour, where transient misreads should not immediately trigger failure if the die is correct.

Performance Logging and Measurement

The framework records:

- Decode latency (milliseconds)
- Payload values
- Timeout events

- Validation outcome

These metrics provide objective evidence of whether the Data Matrix approach satisfies production cycle-time constraints.

4.2. Experimental Setup

4.2.1. Imaging Hardware

The prototype was evaluated using an 8-megapixel USB industrial camera based on the Sony IMX179 CMOS sensor. (same camera used for the placement check)

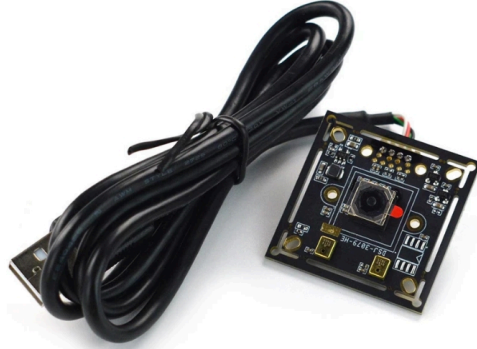


Figure 3: Camera

Camera Specifications

- Sensor: Sony IMX179 CMOS
- Maximum Resolution: 3264×2448 (8 MP)
- Pixel size: $1.4 \mu\text{m}$
- Lens size: $1/3.2''$
- Field of view: 76°
- Dynamic range: 72.5 dB
- Minimum illumination: 0.5 lux
- Interface: USB 2.0 (UVC, driver-free)
- Shutter: Electronic rolling shutter

The camera supports multiple resolution modes up to 3264×2448 @ 15 fps. For evaluation, the system used the highest stable resolution supported by the device (4K) to maximise pixel density over the Data Matrix region.

Image processing and decoding were executed on a LattePanda single-board computer.

4.2.2. Camera Mounting

Compared to the original QAQC placement-check camera position, the camera was repositioned closer to the die face to increase effective pixel density of the Data Matrix symbol.

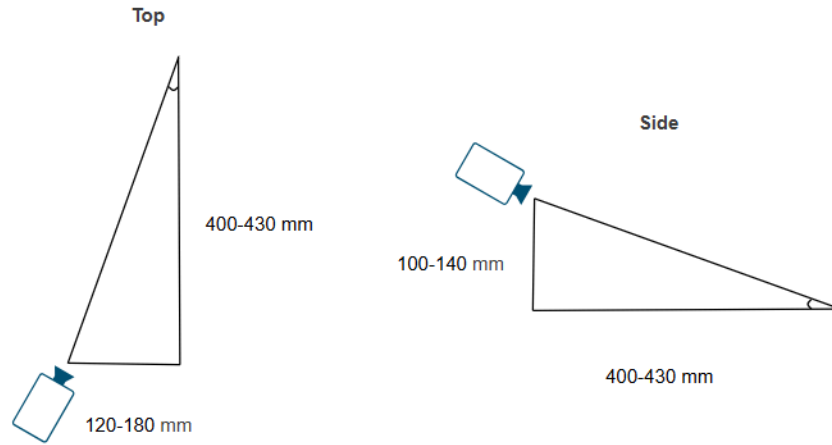


Figure 4: Camera Mounting Position

The working distance between the camera and die surface was adjusted to:

- Maximise symbol pixel footprint
- Maintain full die visibility
- Avoid mechanical interference with operator workflow

4.2.3. Camera Configuration

An ArUco-marker-based region-of-interest (ROI) detection mechanism was used to isolate the die face from the surrounding crimper structure.

- A secondary inner crop of 0.6,
- perspective correction,
- upscaling to 2

were applied to the cropped die image.

To ensure repeatable imaging conditions, automatic camera adjustments were disabled during evaluation. The following parameters were fixed:

- Auto white balance: locked
- Auto focus: locked (focus value = 170)
- Auto exposure: locked (exposure value = 200)

All other parameters (brightness, contrast, saturation, gain, gamma) remained at default values



Figure 5: Camera View

Locking exposure and focus prevented frame-to-frame intensity fluctuation and focus hunting, ensuring consistent pixel characteristics for Data Matrix decoding.

4.2.4. Die and Symbol Configuration

Evaluation was conducted on the 50-size die, selected as the worst-case scenario due to smallest available frontal surface for label placement.

A 6 mm × 6 mm ECC200 Data Matrix symbol was:

- Printed in black on white
- Attached to the front face of all eight die segments
- Secured using transparent adhesive tape

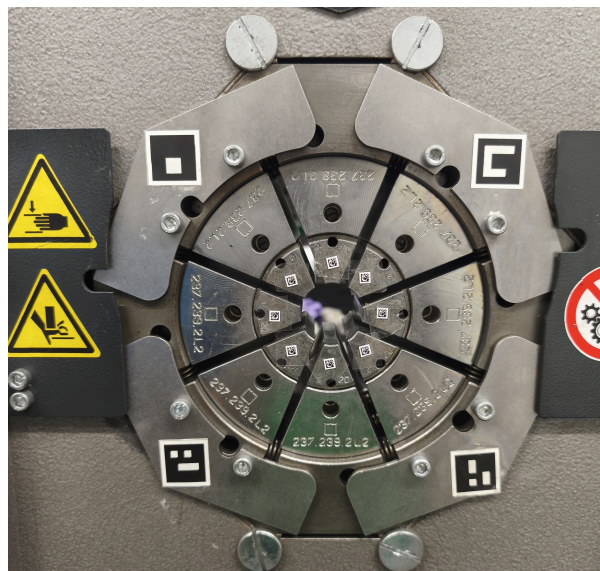


Figure 6: DataMatrix Attached to the Die

All eight die pieces have the DataMatrix symbol attached for redundancy.

4.2.5. Lighting and Environmental Conditions

Testing was conducted in the production room under standard operational lighting conditions.

- Overhead workshop lights were fully active
- No dedicated illumination system was added
- Minor surface oil contamination was present during portions of testing

No environmental conditions were artificially controlled to optimise decoding performance. The evaluation therefore reflects realistic workshop deployment conditions.

4.2.6. Validation Parameters

The live validation framework was configured with the following parameters:

- Decode interval: 250 ms
- Decode timeout: 300 ms
- Cycles per validation attempt: 20
- Required matching decodes for PASS: 3

This results in a maximum theoretical validation duration of: $20 \times 250 \text{ ms} = 5 \text{ seconds}$

4.3. Performance Evaluation

Performance was evaluated using two complementary measurement modes:

1. a fixed-window validation run that executes 20 decode cycles per attempt to characterise decode reliability and error modes, and
2. a live timing mode that terminates early once the validation accumulates three exact payload matches, reflecting the time takes for a PASS validation

4.3.1. Fixed-window decoding reliability

Ten validation attempts were executed on the 50-size die (6 mm ECC200 symbol), each consisting of 20 decode cycles. Across all attempts, the framework achieved a PASS rate of 10/10 (100%). Per-attempt outcomes and supporting statistics are summarised in Table 1.

Table 1: Performance Results per Attempt

Attempt	Matches	NO READ	Wrong reads	Mean decode ms	Result
1	17	2	1	50.8	PASS
2	15	4	1	51.4	PASS
3	12	5	3	58.8	PASS
4	13	2	5	53.0	PASS
5	15	2	3	62.3	PASS
6	16	3	1	60.8	PASS
7	16	4	0	61.3	PASS
8	16	0	4	57.4	PASS
9	15	4	1	60.4	PASS
10	19	1	0	62.8	PASS

At the decode-cycle level, the system produced:

- 77.0% exact-match rate (154 correct payload decodes out of 200 cycles),
- 86.5% read rate (173 non-empty decodes out of 200 cycles),
- 13.5% NO READ rate (27/200 cycles), and
- 9.5% wrong-read rate (19/200 cycles, i.e., a non-target payload returned).

Importantly, no timeout events were observed during the fixed-window run, indicating that the enforced timeout threshold (300 ms) is sufficient for the tested configuration and that the decoder remains responsive under the selected ROI and preprocessing pipeline.

The per-cycle decode time had a mean of ≈ 58 ms, median of ≈ 55 ms, and 95th percentile below ≈ 100 ms. This supports feasibility for integration into the QAQC workflow without introducing perceptible operator delay, provided the camera frame acquisition and ROI extraction remain stable.

4.3.2. Live timing mode

To evaluate workflow-relevant responsiveness, the live validation was executed in “timing mode,” where each run stops as soon as three exact matches are collected (the PASS criterion). The resulting time-to-validate distribution is visualised in Figure 7.

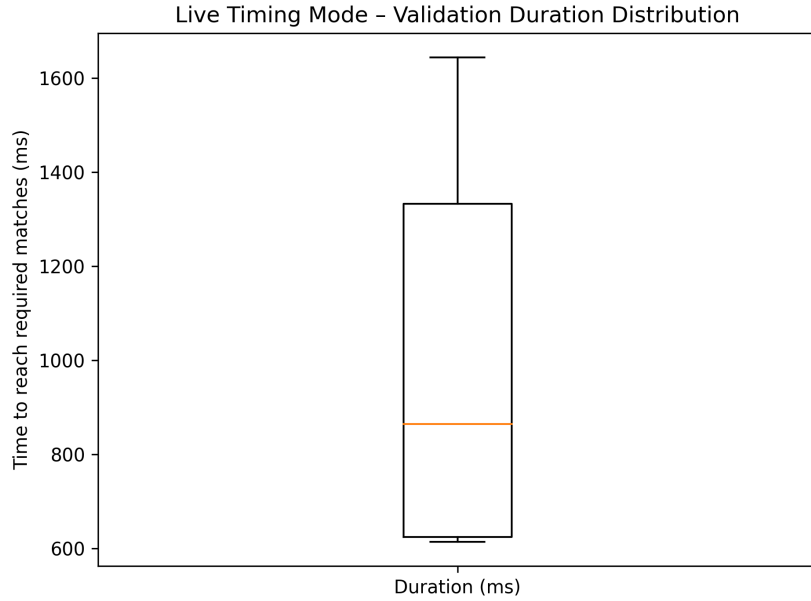


Figure 7: Time to Reach Three Matches

Across 10 timing-mode validations, the time required to reach three matches was:

- Mean: ≈ 0.98 s
- Median: ≈ 0.86 s
- Minimum: ≈ 0.61 s
- Maximum: ≈ 1.64 s

Given the configured decode interval of 250 ms and a maximum window of 20 cycles (5 s theoretical maximum), these results indicate that the validation typically reaches a PASS decision in well under 1 second, and in the worst observed case within ≈ 1.7 seconds. In practical terms, the system can provide PASS/FAIL feedback quickly enough to fit within normal operator workflow, without requiring a prolonged “hold”.

Overall, the combined results demonstrate that Data Matrix decoding on the worst-case die configuration (50-size die with 6 mm code) achieves:

- high decision-level reliability through multi-cycle matching, and
- production-compatible response time when operating in early-stop timing mode.

4.4. Observed Limitations

Although the Data Matrix-based identification approach demonstrated feasibility for die verification under controlled evaluation conditions, several practical limitations were identified during prototype testing.

4.4.1. Camera Mounting Constraints

The camera mounting distance imposes a hard physical limit on decoding reliability. When the camera is positioned too far from the die surface (around the upper bound of the mounting range), the Data Matrix symbol occupies insufficient pixels per module, resulting in:

- Reduced module clarity
- Increased decoding failure

- non-deterministic or incorrect payload reads

Under such conditions, additional preprocessing (e.g., contrast enhancement or dynamic thresholding) does not reliably restore decoding performance, as insufficient spatial resolution cannot be compensated for algorithmically.

Conversely, reducing the mounting distance improves symbol pixel density but may interfere with operator workflow or mechanical clearance, limiting how close the camera can be practically installed.

4.4.2. Industrial Marking Constraints

The prototype evaluation used printed black-on-white symbols attached via adhesive tape. In production deployment, laser etching would likely be required. However, laser-marked codes present additional challenges:

- Lower intrinsic contrast compared to printed black-on-white labels
- Increased specular reflection from metallic surfaces
- Reduced edge sharpness depending on engraving depth

As a result, decoding performance for laser-etched symbols may be inferior to that observed during prototype testing.

Additionally, existing stamped die numbers and surface markings limit available flat surface area for Data Matrix placement. Specific die sizes may provide insufficient space for reliable code engraving.

4.4.3. Additional Manufacturing Cost and Preparation Time

Implementing permanent Data Matrix marking would introduce:

- Additional laser etching cost
- Die preparation time before deployment
- Potential need for re-marking if wear or contamination degrades symbol readability

These factors increase deployment overhead relative to non-invasive identification methods.

4.4.4. Surface Contamination Sensitivity

Reliable decoding requires sufficient contrast between modules and background. Oil residue, dirt, or metal debris on the die surface may reduce symbol clarity. Consequently:

- Operators may need to maintain cleaner die surfaces
- Maintenance requirements may increase compared to the current workflow

This introduces operational dependency on consistent cleaning practices.

4.4.5. QAQC Placement Model Interaction

The existing Placement Check YOLO model was trained using die images without Data Matrix markings. Introducing visible codes on the die surface may alter background features and impact detection performance.

Retraining or fine-tuning of the placement detection model may be required to maintain segmentation and localisation accuracy after Data Matrix integration. Matrix symbol attached for redundancy.

5. Machine Learning Based Die Verification

While the DataMatrix approach provides promising die validation results, it requires close mounting to the crimper, permanent physical marking and introduces practical risks around marking durability, reflectivity, and contamination sensitivity in real workshop conditions.

To complement this, a second prototype pathway was developed using machine learning to verify die identity directly from visual appearance, without requiring modification to existing die sets. In this approach, the system captures an image of the installed die, encodes it into a compact feature embedding, and then verifies the die by comparing the embedding against stored reference embeddings for known die sizes.

The remainder of this section documents the end-to-end ML verification pipeline, including:

- dataset acquisition under controlled variation
- the embedding model design and training procedure
- and live evaluation methodology and decision logic for producing PASS/FAIL/UNCERTAIN outcomes

5.1. Data Collection

5.1.1. Capture Setup and ROI Normalisation

Data was captured using the same camera hardware, mounting position, and working distance as the DataMatrix prototype, ensuring both approaches operate within the same mechanical envelope and share an identical line-of-sight to the die region. The camera is rigidly fixed relative to the die-set location, and all automatic camera controls are disabled to maintain stationary imaging conditions. In particular, exposure is fixed at 200 and focus at 170, with white balance and other acquisition parameters held at their default locked values. This improves reproducibility and prevents the sensor from constantly adjusting the image from frame to frame, which would introduce undesirable non-stationary appearance variation that is detrimental to both dataset consistency and deployment control.

To further reduce sensitivity to operator positioning, each capture is guided by four ArUco markers placed around the die-set region of interest. The four corner points are then ordered consistently (top-left to bottom-left) and mapped via a perspective warp into a fixed-size, square crop (640 px $\times$ 640 px).

(Camera settings are different from DataMatrix setup: no inner crop and upscaling are applied, and a camera resolution of 720p is used instead of 4K.)

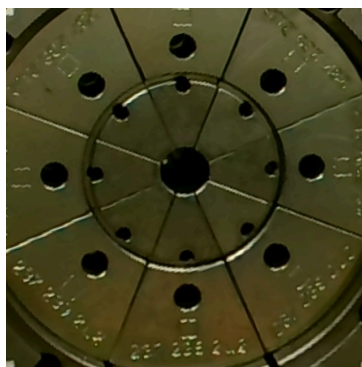


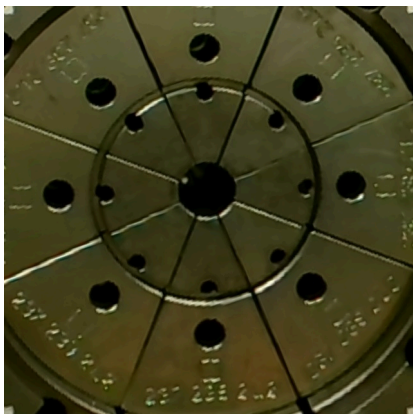
Figure 8: Example Cropped ROI - Die 17

5.1.2. Controlled variation protocol

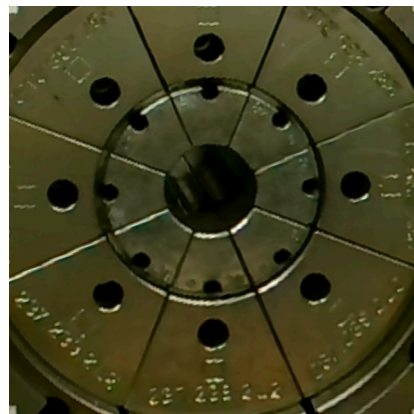
The dataset intentionally includes sets of controlled appearance changes that mirror the dominant sources of embedding drift expected in deployment. This reduces the risk that the model learns spurious cues and improves the reliability of similarity-based verification.

Baseline (nominal operating condition)

- Why: The baseline distribution should reflect the most common workshop state, since it becomes the anchor for both training and subsequent thresholding.
- How it is controlled: Images are collected under standard workshop lighting with the camera in its default mounting position (chosen within the allowable mounting range), and with the die installed in the standard operating configuration. This condition is also used as the reference point against which other variations are compared.



Die 17 baseline



Die 28 baseline

Figure 9: Baseline Variations

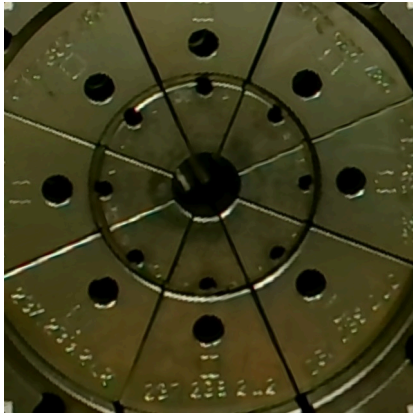
Angle / Distance (viewpoint sensitivity)

- Why: Small viewpoint changes are realistic due to mounting tolerances and operator handling. These changes can alter edge geometry, specular highlights, and the apparent proportions of die features — effects that can shift an embedding even when the die identity is unchanged.
- How it is controlled: The camera is moved only along the permitted mounting path by the existing crimper camera mount, which constrains motion to an arc. This introduces a coupled change in distance and angle: there is no independent adjustment of angle without a change in distance, and there is no change in vertical position.

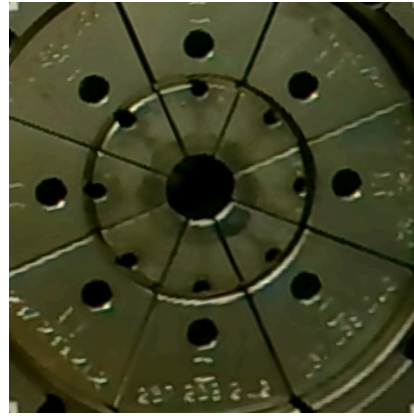
Two bounds define the usable range:

- Near bound: the camera is moved as close as possible while keeping all four ArUco markers entirely in view.
- Far bound: the camera is moved as far as possible before the markers become too small ($< 500px^2$) to pass the acceptance criteria.

By capturing across this arc-limited range, the dataset reflects realistic mounting variability while maintaining reliable ROI extraction.



Die 20 baseline

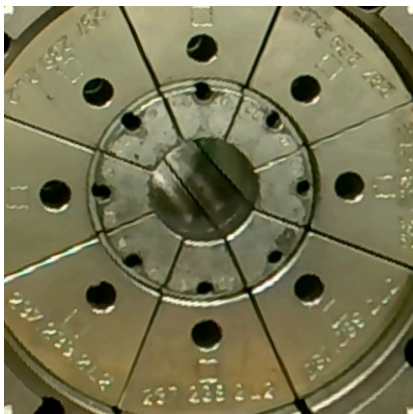


Die 20 angle

Figure 10: Angle Variations

Lighting (photometric sensitivity)

- Why: Workshop lighting can vary across time, location, and surrounding equipment, and dies can present strong specular components. Explicitly sampling lighting variation reduces the risk of the model overfitting to a single brightness/contrast regime.
- How it is controlled: Illumination changes are induced by adjusting camera exposure time across a defined range (100-700). Because a single switch controls the production room lighting, exposure provides a clean, controllable way to vary image brightness without introducing additional lighting sources.



Die 32 lighting



Die 32 lighting

Figure 11: Lighting Variations

Dirty / contaminated surface (material and reflectivity change)

- Why: In practice, dies accumulate grease and residue, which modifies texture visibility and increases glare. These effects can be more disruptive to appearance than a mild pose change, so they must be represented explicitly.
- How it is controlled: Grease is deliberately applied to the die surface to create a repeatable “dirty” condition. Captures are taken after application using the same framing and ROI normalisation as the baseline, ensuring the model learns to tolerate contamination rather than treating it as a class cue.

Rotation (in-plane orientation)

- Why: The die may be installed with different rotational alignment, and similarity-based verification should not depend on a single orientation. Including rotations reduces orientation bias and encourages the embedding to encode die-specific structure over absolute orientation. (Rotation can be done in data augmentation, but since the mother die is included in the captured image, the rotation variation shows the rotation relative to the mother die.)
- How it is controlled: The die is rotated in place within the mother die, keeping the overall setup unchanged while varying only the in-plane orientation of the die features relative to the camera.

Other sources of variability during capture was shadowing (e.g., standing in the light path or a hand briefly occluding the illumination) and die openness (some samples are fully closed, while others are slightly open). Although operators are instructed to close the die before validate, achieving perfectly consistent closure is not always practical in workshop conditions. These effects were not treated as standalone variation categories; instead, they are implicitly captured within the existing variation sets.

Overall, these variations were selected because they represent the most plausible, high-impact sources of real-world appearance change, while still being controllable enough to preserve dataset structure and interpretability during training and evaluation.

5.1.3. Dataset Splitting and Closed/Open-Set Configuration

To obtain a meaningful estimate of deployment performance, the dataset was split by capture session rather than by randomly shuffling individual images. Session-based splitting reduces leakage from near-duplicates (e.g., consecutive frames with near-identical lighting, alignment, and surface state) and better reflects the true generalisation requirement: the model must remain stable across different collection runs where nuisance factors drift naturally (operator handling, ambient lighting, minor setup tolerances), even though the camera rig and ROI normalisation remain fixed.

Training/Validation/Test split

Two independent sessions were assigned to the training set to increase diversity and volume, while validation and test each use their own held-out session to measure generalisation to unseen capture conditions. The planned image counts per split are controlled via per-die quotas for each variation type, ensuring balanced coverage across nuisance factors.

The table below summarises the total number of images per split and variation for the closed-set die classes used in training and validation (17, 20, 28, 32, 36, 40, 44, 50):

Table 2: Training/Validation/Test split

Split	Ses- sion(s)	# Dies	Base- line / die	Angle / die	Light- ing /die	Clean/ Oily / die	Rota- tion / die	Total / die
Train	s1, s2	8	28	12	12	10	8	70
Valida- tion	s3	8	5	3	3	2	2	15
Test	s4	8	5	3	3	2	2	15

Closed-set vs Open-set dies

Closed-set: 17, 20, 28, 32, 36, 40, 44, 50.

These represent the primary die sizes of interest for learning a discriminative embedding space and for standard closed-set verification performance measurement.

Open-set: 24 and 57.

Both are used in the assembly process, but they have distinct physical appearance compared with the standard set: the 24 die sits offset relative to the mother-die front face, and the 57 configuration has the die piece and mother die combined as a single unit, requiring the mother die to be replaced. Because these configurations can shift the visual distribution in ways not seen in the closed-set classes, they are ideal for evaluating whether the verification method can:

- reject “unknown” dies rather than forcing an incorrect match
- scale to new die variants without retraining from scratch.

A smaller dedicated set was also captured for the open-set dies (24 and 57) to build their reference embeddings. The per-die counts for this reference set are:

Table 3: Open-set Die Collection

Die	Baseline	Angle	Lighting	Clean/Oily	Rotation	Total
24	14	6	6	5	4	35
57	14	6	6	5	4	35

Important Note

Due to project time constraints, the full capture plan was not completed. In Table 2 (closed-set), only the cells highlighted in yellow were collected and used for model training and validation; a dedicated test split was not collected. For the open-set dies (Table 3), only the yellow cells were collected, and these samples were used solely for reference image/embedding building.

5.1.4. Dataset Structure

The dataset was organised using a structured directory hierarchy compatible with PyTorch’s ImageFolder format. This structure enables automatic class indexing and simplified data loading during training and validation.

The dataset root directory is divided into separate splits:

```

data_root/
├── train/
│   ├── die_17/
│   ├── die_20/
│   ├── die_24/
│   └── ...
└── val/
    ├── die_17/
    ├── die_20/
    ├── die_24/
    └── ...

```

5.2. Model and Training

This work frames die validation as a verification problem (is this die the claimed die?) rather than a closed-set multi-class classification problem. Instead of predicting a die label directly, the system learns an embedding function that maps each cropped die ROI image into a compact feature vector. Verification is then performed by comparing the embedding of a live image to stored reference embeddings (class centroids) using cosine similarity.

5.2.1. Model Choice

Model Backbone

The training pipeline uses an EfficientNet-B0 backbone as the feature extractor, paired with a small projection head to produce fixed-length embeddings for verification. EfficientNet-B0 was selected because it provides a strong accuracy-to-compute trade-off: it is lightweight enough to be practical for iterative experimentation and CPU-friendly deployment constraints, while still offering sufficient representational capacity to separate visually similar die classes.

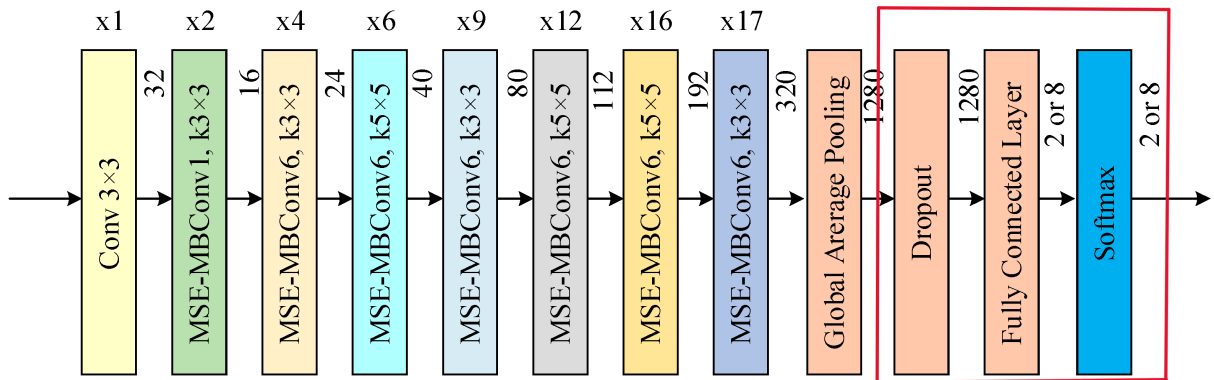


Figure 12: EfficientNet-B0 with Classifier Head

The backbone is initialised with ImageNet-pretrained weights rather than training from scratch. This improves convergence speed and stability on a relatively small, domain-specific dataset, and provides robust low-level features (edges, corners, texture primitives) that remain useful for die imagery. Fine-tuning then adapts these generic features to the die-specific visual cues needed for verification.

Input Layer

The network operates on 3-channel (RGB) images resized to a fixed square resolution (224×224). Before being passed to the network (during both training and inference), images are converted to tensors and normalised using the standard ImageNet mean and standard deviation, matching the backbone’s pretraining assumptions.

Output Layer

Rather than using the backbone’s default classification output, the architecture is configured to return pooled feature vectors (global average pooled feature). These features are passed through a lightweight embedding head consisting of:

- a linear projection from backbone feature dimension to a 256-D embedding, followed by
- batch normalisation to improve feature scaling and training stability.

Finally, the embedding is L2-normalised to unit length. This design makes cosine similarity meaningful and stable at inference time, because distances depend on angular separation in the embedding space rather than raw feature magnitude.

5.2.2. Loss Function

To train a verification-style embedding, the model uses an ArcFace-style angular margin loss implemented as an auxiliary ArcFace classification head attached after the embedding is L2-normalised.

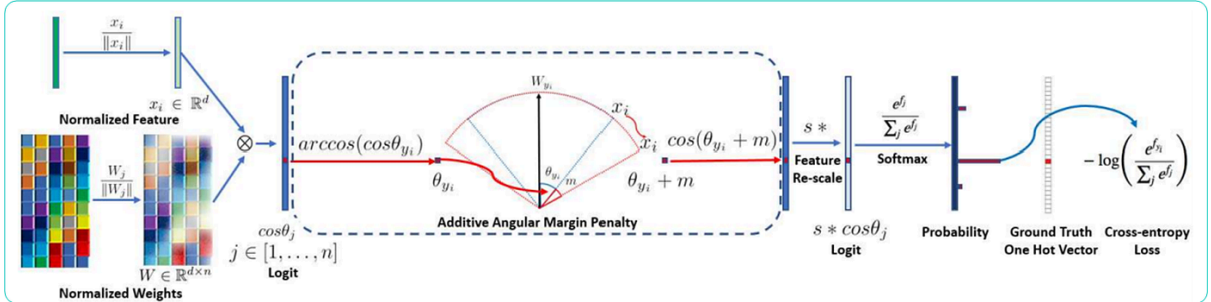


Figure 13: ArcFace Loss

ArcFace head (training-only classifier)

The ArcFace head can be viewed as a “training-time classifier” whose job is not to be deployed, but to shape the embedding space. It keeps a learnable weight vector for each class (each die size), and because both the embedding and the class weights are normalised, the classifier score is based on the cosine of the angle between them:

$$\cos(\theta) = x^T w$$

Instead of only encouraging “correct class has the highest score” (as in a standard softmax head), ArcFace modifies the target class score by adding an angular margin m . Therefore, even if a sample is already classified correctly, ArcFace still pushes it to be more confidently inside its class region by enforcing an explicit gap between classes in angular space. The head also applies a scale factor s to keep optimisation stable:

$$\text{logit}_y = s \cos(\theta_y + m)$$

All other class logits remain based on $\cos(\theta)$.

Training-only note: This ArcFace head is only used during training to encourage a well-structured embedding space. During inference, the system does not rely on this classifier head; it uses the backbone embedding directly and compares it against stored class representations (centroids) using cosine similarity.

ArcFace loss function

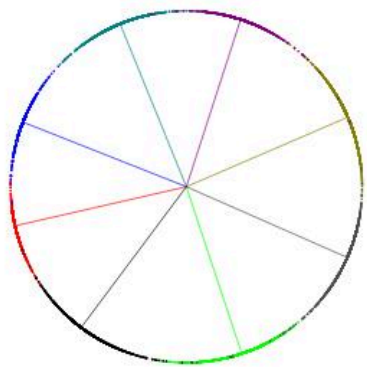
Once these ArcFace-adjusted logits are produced, the optimisation still uses a standard cross-entropy loss, but on logits that incorporate the angular margin. So while the loss form is familiar, the geometry it enforces is different: it explicitly trains embeddings to be compact within a class and separated between classes by a margin in angular space.

Arcface Verification vs Classification

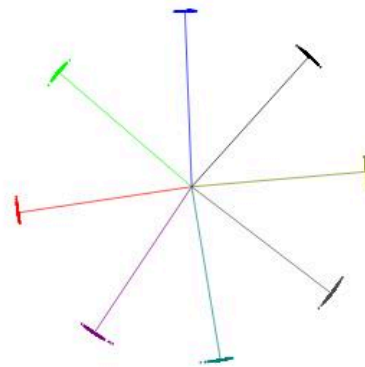
A default softmax classification head is typically sufficient when classes are visually distinct. However, in this project the hardest failure mode is adjacent die sizes: dies that differ only slightly in inner diameters can look nearly identical in cropped images, making them easy to confuse.

ArcFace is better suited to this scenario because it:

- Reduces adjacent-class confusion by enforcing a fixed separation margin between classes in the embedding space.
- Produces tighter clusters for each die size, which improves robustness to small changes in pose, lighting, and cropping.
- Optimises embeddings in a way that directly matches the downstream verification step, where decisions are based on cosine similarity / nearest centroid.



(a) Softmax



(b) ArcFace

Figure 14: Softmax vs ArcFace Embedding Space

Overall, ArcFace turns the training objective into “make die-size identities separable with a guaranteed angular gap,” which is exactly what is needed when two neighbouring classes are extremely similar visually.

5.2.3. Data Augmentation

The chosen augmentation set is deliberately conservative: it targets realistic capture variability (small framing / rotation / lighting shifts) while avoiding transformations that would either distort geometry or erase fine details that the verification embedding needs to retain.

Table 4: Data Augmentations

Augmentation	Parameters used	Why it helps for die-size verification
Resize	Resize to $\approx 1.15 \times$ input size	Gives the crop step room to jitter without losing content; reduces overfitting to one exact framing.
RandomResizedCrop	Random crop back to 224×224 with small zoom (90-100%) and near-square aspect (0.98-1.02)	Makes embeddings robust to small ROI extraction misalignment while preserving geometry; tight ratio range avoids unrealistic stretching.
RandomRotation	Random in-plane rotation up to $\pm 5^\circ$	Encourages invariance to small orientation shifts without creating unrealistic views that could remove edge cues.
ColorJitter (brightness, contrast)	Brightness and contrast jitter up to $\pm 10\%$	Stabilises embeddings under illumination changes while keeping jitter mild so fine-grained texture/edge differences aren't washed out.

5.2.4. Training Strategies

Transfer Learning and Backbone Freezing

The embedding model utilised a pretrained EfficientNet backbone. A two-stage fine-tuning strategy was adopted:

- Initial Freezing Phase. During the first N epochs, backbone parameters were frozen and only the projection head and ArcFace classifier were trained.
- Full Fine-Tuning Phase. After the freezing period, the backbone was unfrozen and the entire model was fine-tuned end-to-end.

This staged approach provides several benefits:

- Preserves pretrained low-level visual features.

- Stabilises early angular-margin optimisation.
- Reduces overfitting risk in small datasets.

Balanced Batch Sampling

To improve metric learning performance, an optional M-per-class balanced sampling strategy was implemented.

Instead of randomly sampling images, each batch consisted of:

- M distinct die classes
- K samples per class
- Batch size = $M \times K$

This approach ensures:

- Balanced class representation per batch.
- Strong intra-class comparison signals.
- Reduced dominance of frequent classes.
- Improved angular margin optimisation stability.

Optimisation Strategy

Model optimisation was performed using:

- AdamW optimiser
 - Incorporates decoupled weight decay.
 - Provides improved generalisation compared to standard Adam.
- Cosine Annealing Learning Rate Scheduler
 - Gradually reduces the learning rate.
 - Improves convergence stability.
 - Reduces oscillation near local minima.

Weight decay was applied as a regularisation mechanism to reduce overfitting.

5.2.5. Validation and Checkpoint Selection

Unlike conventional classification training, the model was not selected based on classification accuracy.

The primary selection metric was:

- True Acceptance Rate (TAR) at a fixed False Acceptance Rate (FAR)

Centroid Computation

At each validation stage, class reference embeddings (centroids) are computed using all training images without augmentation. For each die class, embeddings are averaged and L2-normalised to form a stable prototype.

These centroids serve as reference representations during validation and deployment.

Similarity Evaluation

For each validation image:

- An embedding is generated.

- Cosine similarity is computed against all class centroids.

This produces:

- A positive score (similarity to the correct centroid).
- Multiple negative scores (similarity to incorrect centroids).

Hard Negative Construction

To simulate realistic industrial confusion, validation focuses on hard negatives:

- For each sample, the top-K highest similarities among incorrect classes are selected.
- These represent the most visually similar wrong die sizes.

This evaluation is more stringent than using all negatives and better reflects real-world risk.

AR@FAR Metric

Model performance is measured using True Acceptance Rate (TAR) at a fixed False Acceptance Rate (FAR).

For a chosen FAR:

- A threshold τ is computed from negative similarity scores such that:

$$\text{FAR} = P(s_{\text{negative}} \geq \tau)$$

- TAR is then calculated as:

$$\text{TAR} = P(s_{\text{positive}} \geq \tau)$$

This ensures the model is evaluated under a controlled false acceptance constraint.

Best Model Selection

During training, validation is performed at fixed intervals. The best checkpoint is selected according to:

$$\text{Best Model} = \operatorname{argmax} \text{TAR}_{\text{hard}}(\text{FAR} = \alpha)$$

where:

- α is the predefined FAR target,
- Evaluation is performed using hard negatives.

This strategy ensures that model selection directly aligns with the industrial requirement of minimising incorrect die acceptance while maximising correct validation.

After training:

- The best checkpoint is reloaded.
- Final centroids are recomputed.
- Thresholds corresponding to each predefined FAR are exported.
- A deployment-ready model bundle is generated.

5.2.6. Validation Performance (Best Training Run)

Training Configuration

Table 5: Training Configuration

Category	Parameter	Value
Training Duration	Epochs	30
	Freeze Epochs	3
Optimisation	Optimiser	AdamW
	Learning Rate	0.0003
	Weight Decay	0.0003
	Scheduler	Cosine Annealing
Metric Learning (ArcFace)	ArcFace Margin	0.35
	ArcFace Scale	35.0
Batching Strategy	Balanced Sampler	True
	M Classes per Batch	8
	K Samples per Class	4
	Batches per Epoch	80
Validation Settings	Evaluation Frequency	Every 1 epoch
	Hard Negative Top-K	4
	Checkpoint FAR Target	0.01

Nearest-Centroid Accuracy

The predicted class is assigned as the centroid with the highest similarity (i.e., nearest in embedding space).

The final nearest-centroid validation accuracy was:

95.31%

This metric indicates strong class separability but was not used as the primary model selection criterion.

Verification Performance (TAR@FAR)

Performance was evaluated using True Acceptance Rate (TAR) at fixed False Acceptance Rates (FAR). Results are shown below.

Hard Negatives (Primary Selection Metric)

Table 6: TAR@FAR for Hard Negatives

FAR	TAR
0.10	100.0%
0.05	98.44%
0.01	95.31%

At the strict operating point of FAR = 1%, the model achieves: 95.31% True Acceptance Rate. This means that under a controlled 1% false acceptance constraint, over 95% of correct die selections are validated successfully.

All Negatives (Optimistic Baseline)

Table 7: TAR@FAR for All Negatives

FAR	TAR
0.10	100.0%
0.05	100.0%
0.01	96.88%

These values are slightly higher because all negatives include easier, visually distinct die sizes.

Selected Operating Threshold

For deployment at FAR = 0.01, the cosine similarity threshold using hard negatives is:

$$\tau = 0.4293$$

This threshold is exported and used directly in the live validation system.

Checkpoint Selection

The best model was selected using:

$$\operatorname{argmax} \operatorname{TAR}_{\text{hard}}(\text{FAR} = 0.01)$$

Hard negatives were used because they represent the most challenging misclassification cases. Selecting the checkpoint based on this metric ensures that the deployed model is optimised for minimising incorrect die acceptance.

The optimal checkpoint was identified at epoch 6, where the highest TAR at FAR = 0.01 was achieved (95.31%). Beyond this point, further training did not yield measurable improvements in TAR@FAR performance, indicating that the model had effectively converged.

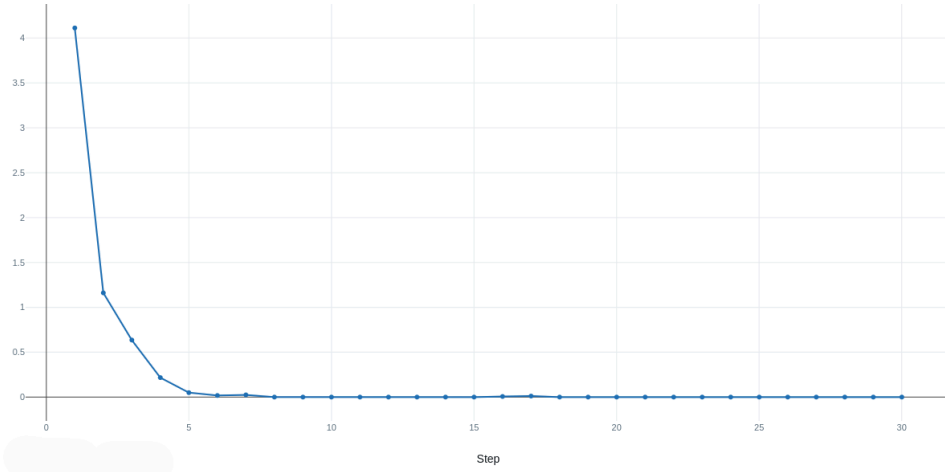


Figure 15: Train Loss

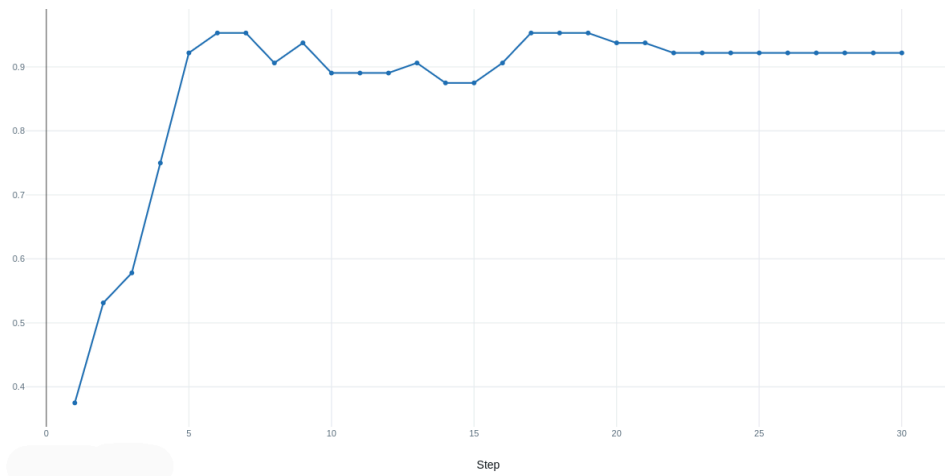


Figure 16: Checkpoint TAR@FAR 0.01

5.3. Performance Evaluation

Due to internship time constraints, a dedicated test-set image split was not collected for the ML approach, so no offline “test set” performance is reported. Instead, performance is evaluated through live verification trials using planned recommended vs installed die pairs that simulate the real die validation workflow.

5.3.1. Live Test Setup

Hardware and imaging configuration

Live evaluation was conducted using the same imaging hardware and physical setup established for the DataMatrix prototype evaluation and the ML dataset collection stage. In particular, the system uses the same fixed camera hardware, with a rigid mounting relative to the die-set location, and the same mounting position and working distance so that performance differences observed during evaluation are attributable to the verification method rather than changes in viewpoint or optics. The camera is operated under locked, repeatable settings (auto controls disabled) and the die region is extracted using the same ArUco-guided ROI normalisation pipeline: four markers define a quadrilateral around the die area, which is then perspective-warped into a fixed-size square crop.

Planned die-pair test set

Close-set testing (30 pairs):

To evaluate live verification behaviour under both expected and stress conditions, planned tests were constructed as recommended vs installed die pairs across three categories:

- Positive pairs (correct die installed, 10 pairs)

These trials measure true-accept performance when the installed die matches the recommended specification. The closed-set plan includes positives for the trained die sizes: die_17, die_20, die_28, die_32, die_36, die_40, die_44, die_50.

- Hard-negative pairs (adjacent / visually similar mismatches, 10 pairs)

These trials target the most operationally risky confusions — primarily adjacent sizes that are visually similar and therefore most likely to produce high similarity scores. The planned hard-negative set includes swaps such as $17 \leftrightarrow 20$, $22 \leftrightarrow 28$, $28 \leftrightarrow 32$, $32 \leftrightarrow 36$, $36 \leftrightarrow 40$, and $44 \leftrightarrow 50$.

- Easy-negative pairs (visually distinct mismatches, 10 pairs)

These trials cover broader separation mismatches intended to confirm that obviously different dies are rejected reliably (or fail correctly under the decision logic). Examples include $17 \leftrightarrow 50$, $20 \leftrightarrow 44$, $28 \leftrightarrow 50$, $36 \leftrightarrow 17$, and $32 \leftrightarrow 44$.

Open-set testing (20 pairs):

In addition to the closed-set plan, live evaluation includes a dedicated open-set test set targeting die sizes that were not used during model training. This set covers die_24 and die_57, and evaluates each one against the full die pool available during testing (17, 20, 28, 32, 36, 40, 44, 50, 24, 57).

For each open-set recommended die (24 and 57), the plan consists of 10 pairs:

- 1 positive pair where the installed die matches the recommendation ($24 \leftrightarrow 24$, $57 \leftrightarrow 57$).
- 9 negative pairs where the installed die is any other die in the pool (e.g., $24 \leftrightarrow 17$, $24 \leftrightarrow 20$, ..., $24 \leftrightarrow 57$ and $57 \leftrightarrow 17$, $57 \leftrightarrow 20$, ..., $57 \leftrightarrow 57$).

Model and Centriods

The live evaluation used the best-performing training checkpoint selected at the strict operating point of $\text{TAR@FAR} = 0.01$ (hard-negative evaluation), which achieved 95.31% TAR at FAR = 1% and exports the corresponding cosine-similarity operating threshold for deployment. For verification, each closed-set die class is represented by a centroid embedding computed by averaging (and L2-normalising) embeddings over the full closed-set training images, producing one stable prototype per die; in this deployment configuration, each closed-set centroid was built from 52 training images per die. Open-set dies die_24 and die_57 are handled by constructing additional reference centroids from smaller, dedicated capture sets collected for each die, with 26 images per die used to form each open-set centroid.

5.3.2. Decision Logic

The live validation logic operates as a 3-state verifier (PASS / FAIL / UNCERTAIN) over a short capture window, using centroid similarity and conservative voting.

Per-frame similarity scoring

For each valid frame, the cropped ROI is embedded by the deployed model, and cosine similarity is computed against every stored die centroid (closed-set centroids, plus open-set centroids). This yields:

- s_{rec} : similarity to the recommended die centroid
- s_{top1} : maximum similarity across all centroids (top-1 match)

Per-frame 3-state labelling (threshold + margin)

Each valid frame is labelled using a cosine similarity threshold τ (exported from the best training run) and an optional separation margin:

- **PASS** if the recommended die matches confidently:

$$s_{\text{rec}} \geq \tau$$

- **FAIL** if a different die matches confidently and is clearly better than the recommended:

$$s_{\text{top1}} \geq \tau \text{ and } (s_{\text{top1}} - s_{\text{rec}}) \geq \text{margin}$$

- **UNCERTAIN** if no die matches confidently (insufficient evidence):

$$s_{\text{top1}} < \tau$$

where the top-1 centroid corresponds to a class not equal to the recommended die. If the top-1 die exceeds τ but the margin condition is not met, the frame is treated conservatively as UNCERTAIN

Session-level decision (majority vote)

Validation is run over a fixed 1 second window, producing multiple per-frame labels. The final session result is decided by a majority vote over PASS/FAIL/UNCERTAIN counts:

- The winning class must be strictly greater than the other two.
- Any tie results in UNCERTAIN.

5.3.3. Results

Closed-set (dies 17, 20, 28, 32, 36, 40, 44, 50)

Across all 30/30 planned trials (10 positive, 10 hard-negative, 10 easy-negative), the system produced the expected session-level decision outcome: all positive pairs returned PASS and all negative pairs returned FAIL.

No session-level UNCERTAIN outcomes occurred, indicating that similarity evidence was consistently strong enough to produce decisive results under the chosen thresholding and voting logic.

Frame-level ratios showed that negative cases were dominated by FAIL-labelled frames; a small number of trials contained minor ambiguity at the frame level (occasional UNCERTAIN frames for some easy negatives, and one hard-negative case with a small fraction of PASS-labelled frames), but the majority vote remained correct in all cases.

Pair #	Type	Recommended	Installed	Decision	Valid frames	Pass ratio	Fail ratio	Uncertain ratio	Time of Evaluation (ms)	Outcome
1	positive	die_17	die_17	PASS	7	1	0	0	1085	Correct
2	positive	die_20	die_20	PASS	7	1	0	0	1088	Correct
3	positive	die_28	die_28	PASS	7	1	0	0	1097	Correct
4	positive	die_32	die_32	PASS	7	1	0	0	1094	Correct
5	positive	die_36	die_36	PASS	7	1	0	0	1086	Correct
6	positive	die_40	die_40	PASS	7	1	0	0	1094	Correct
7	positive	die_44	die_44	PASS	7	1	0	0	1086	Correct
8	positive	die_50	die_50	PASS	7	1	0	0	1096	Correct
9	positive	die_32	die_32	PASS	7	1	0	0	1089	Correct
10	positive	die_40	die_40	PASS	7	1	0	0	1082	Correct
11	hard_negative	die_17	die_20	FAIL	7	0	1	0	1093	Correct
12	hard_negative	die_20	die_17	FAIL	7	0	1	0	1084	Correct
13	hard_negative	die_20	die_28	FAIL	7	0	1	0	1089	Correct
14	hard_negative	die_28	die_32	FAIL	7	0	1	0	1082	Correct
15	hard_negative	die_32	die_28	FAIL	7	0	1	0	1085	Correct
16	hard_negative	die_32	die_36	FAIL	7	0	1	0	1090	Correct
17	hard_negative	die_36	die_40	FAIL	7	0.14	0.86	0	1089	Correct
18	hard_negative	die_40	die_36	FAIL	7	0	1	0	1077	Correct
19	hard_negative	die_44	die_50	FAIL	7	0	1	0	1089	Correct
20	hard_negative	die_50	die_44	FAIL	7	0	1	0	1084	Correct
21	easy_negative	die_17	die_50	FAIL	7	0	1	0	1081	Correct
22	easy_negative	die_50	die_17	FAIL	7	0	0.86	0.14	1093	Correct
23	easy_negative	die_20	die_44	FAIL	7	0	1	0	1083	Correct
24	easy_negative	die_44	die_20	FAIL	7	0	1	0	1089	Correct
25	easy_negative	die_28	die_50	FAIL	7	0	1	0	1077	Correct
26	easy_negative	die_36	die_17	FAIL	7	0	0.71	0.29	1092	Correct
27	easy_negative	die_40	die_17	FAIL	7	0	0.86	0.14	1102	Correct
28	easy_negative	die_32	die_44	FAIL	7	0	1	0	1094	Correct
29	easy_negative	die_44	die_32	FAIL	7	0	1	0	1093	Correct
30	easy_negative	die_50	die_28	FAIL	7	0	1	0	1083	Correct

Figure 17: Close Set Test Results

Open-set (dies 24 and 57).

Both open-set positive trials returned PASS. For the 18 negative trials, 17/18 returned the expected FAIL decision on the first attempt. The only incorrect outcome occurred for recommended = die_57 with installed = die_28, where the session returned PASS due to a split vote at the frame level (PASS ratio 0.57 vs FAIL ratio 0.43), indicating a borderline case near the operating threshold rather than a consistently wrong match.

Importantly, when the same pair was validated a second time, the system returned FAIL (correct), suggesting this confusion is intermittent and sensitive to short-term appearance variation (e.g., micro-pose, reflections, or ROI stability) rather than a systematic inability to separate the two dies.

The open-set results show the model can separate untrained die classes once a reference centroid is provided – supporting the intended scalability benefit that new dies can be introduced without retraining the embedding model, by collecting a small reference set to build a centroid. In this prototype, 26 images per new die (as used for die_24 and die_57) was sufficient and can be captured in under 20 minutes in the current setup.

Pair #	Recommended	Installed	Decision	Valid frames	Pass ratio	Fail ratio	Uncertain ratio	Time of Evaluation (ms)	Outcome
1	die_24	die_24	PASS	7	1	0	0	1098	Correct
2	die_24	die_17	FAIL	7	0	0.57	0.43	1085	Correct
3	die_24	die_20	FAIL	7	0	1	0	1086	Correct
4	die_24	die_28	FAIL	7	0	1	0	1094	Correct
5	die_24	die_32	FAIL	7	0	1	0	1089	Correct
6	die_24	die_36	FAIL	7	0	1	0	1074	Correct
7	die_24	die_40	FAIL	7	0	1	0	1092	Correct
8	die_24	die_44	FAIL	7	0	1	0	1095	Correct
9	die_24	die_50	FAIL	7	0	1	0	1087	Correct
10	die_24	die_57	FAIL	7	0	1	0	1099	Correct
11	die_57	die_57	PASS	7	1	0	0	1097	Correct
12	die_57	die_17	FAIL	7	0	1	0	1084	Correct
13	die_57	die_20	FAIL	7	0	1	0	1098	Correct
14	die_57	die_28	PASS	7	0.57	0.43	0	1088	Incorrect
15	die_57	die_32	FAIL	7	0	1	0	1089	Correct
16	die_57	die_36	FAIL	7	0	1	0	1102	Correct
17	die_57	die_40	FAIL	7	0	1	0	1097	Correct
18	die_57	die_44	FAIL	7	0	1	0	1094	Correct
19	die_57	die_50	FAIL	7	0	1	0	1095	Correct
20	die_57	die_24	FAIL	7	0	1	0	1090	Correct

Figure 18: Open Set Test Results

Overall, the live trials demonstrate consistently high validation accuracy across both closed-set and open-set dies, with session decisions typically reached within an approximately 1-second validation window. This indicates the embedding + centroid approach can deliver fast, reliable PASS/FAIL feedback under realistic capture conditions, supporting its suitability as a practical die validation mechanism for integration into the QAQC workflow.

6. QAQC Integration Design and Implementation

6.1. Approach Selection

Two candidate approaches were validated as technically feasible for die verification:

- DataMatrix decoding and
- ML-based visual verification using embedding similarity.

Both can produce reliable PASS/FAIL outcomes when the installed die is imaged and compared against the required die for the current assembly context.

However, the ML approach was selected for QAQC integration because the DataMatrix pathway introduces several deployment constraints identified in Section 4.4. In particular, DataMatrix decoding is tightly bounded by camera mounting distance and pixel density — moving the camera far enough to satisfy mechanical clearance and operator workflow can reduce module resolution and degrade decoding reliability in a way that preprocessing cannot consistently recover. In production, DataMatrix would also require permanent laser etching, which typically has lower contrast than printed labels and is more susceptible to specular reflection and edge softness; additionally, some dies may not provide sufficient flat area due to existing markings. This pathway further increases manufacturing overhead (etching cost, preparation time, potential re-marking) and places higher dependency on die cleanliness, since oil/debris can reduce symbol contrast and increase maintenance burden. Finally, intro-

ducing visible codes can interfere with the existing Placement Check YOLO model (trained on unmarked dies), potentially requiring retraining or fine-tuning to preserve localisation performance — adding risk and scope to integration.

By contrast, the ML verification pipeline achieves die validation without physically modifying die sets, directly addressing the marking/cost/maintenance risks above. It also aligns cleanly with the existing imaging envelope (same camera hardware and mounting assumptions) and is designed to tolerate realistic appearance variation through ROI normalisation and controlled variation coverage (pose, lighting, contamination).

As a result, the embedding + centroid approach was chosen as the primary integration path for the QAQC system, with DataMatrix retained as a viable alternative where marking and mounting constraints can be guaranteed.

6.2. Integration Point

The die validation module was integrated directly into Step 3 (Placement Check) as a gate condition rather than implemented as a standalone QAQC step. This decision was driven primarily by process logic. Die selection is a prerequisite to correct crimping: if the wrong die set is installed, the crimp diameter will be incorrect regardless of hose placement accuracy. Since Step 3 is the first stage where the hose assembly is positioned inside the crimper, it represents the natural control point to verify die correctness before hydraulic engagement. Integrating die validation here ensures that incorrect die selection is intercepted before irreversible deformation occurs.

Additionally, because die validation is handled internally within Step 3, no additional workflow index or scanning logic was required in the step progression mechanism. This preserves backward compatibility while extending system capability.

6.3. Database Changes

Table 8: Columns Added to the Database

Column Name	Data Type	Description
last_validated_die (qaqc_branches)	Integer	Stores the most recently validated die size for a branch session. Used to determine whether die validation is required again when processing subsequent assemblies.
dieset_value (qaqc_assemblies)	String	Records the detected or confirmed die size during validation. Provides traceability between the recommended die and the physically installed die.
dieset_passed (qaqc_assemblies)	Boolean	Indicates whether the die validation check passed (True) or failed (False).
dieset_img (qaqc_assemblies)	String	Stores the reference to the captured validation image (URL). Enables trace-back of the validation result.

Note

The die validation model and die reference centroids are not stored in the database. Instead, they are stored as deployment artifacts within a dedicated artifacts directory on the backend server.

6.4. Step 3 Flag Changes

Table 9: Step 3 Backend PKG

Field	Type	Description
placement	String	Unchanged
valid_flag	Boolean	Unchanged
passed_flag	Boolean	Unchanged
finish_flag	Boolean	Unchanged
is_placement_flag	Boolean	Mode indicator. false = die validation mode, true = placement mode. Used by the UI to render the right controls.
engage_pixel_not_found_flag	Boolean	true when engaged-pixel calibration data is missing (backend forces the entire Step 3 to fail and finish).
die_passed_flag	"PASS" "FAIL" "UNCERTAIN" "NULL"	Die validation state. Updated when a die validation attempt is made; "NULL" when not yet evaluated.
system_error	String	Populated only if an internal exception occurs; allows UI to show a system-level error message.

Table 10: Step 3 Frontend PKG

Field	Type	Description
email	String	Unchanged
error	String	Deleted
die_validation_flag	Boolean	Operator action to trigger die validation evaluation while in die validation mode (“Validate” button).
die_skip	String	Operator override to skip die validation. If a string is provided, it is logged as the skip reason; otherwise a default reason is used.
place_skip	String	Operator override to skip placement check. If a string is provided, it is logged as the skip reason; otherwise a default reason is used.

6.5. Die Validation Workflow

6.5.1. High Level Overview

Die validation is executed as a pre-check within Step 3 (Placement Check) to confirm that the operator has installed the correct die set before crimping begins. When Step 3 is initiated, the backend receives the assembly context (job ID, assembly ID, hose end index) together with the recommended die size for that hose end. The system first determines whether die validation is required. This decision is based on two conditions: whether die validation has already been completed for the current hose end, and whether the branch’s `last_validated_die` matches the recommended die size. If validation is not required, the workflow proceeds directly to placement evaluation. Otherwise, the system enters die verification mode and prompts the operator to confirm or adjust the installed die.

In die verification mode, the backend continuously streams frames from the crimper camera and waits for an explicit validation trigger from the frontend (when the operator presses the “Validate” button). Upon receiving this trigger, the backend executes the die validation inference pipeline on the captured camera frames and compares the detected die against the recommended specification. The result is returned to the frontend as `die_passed_flag`, which can be PASS, FAIL, or UNCERTAIN. The user interface uses this status to guide operator action. In cases of FAIL or UNCERTAIN, the operator is prompted to replace the die with the correct one or improve visibility (e.g., clean the die surface) before retrying validation. Only when the result is PASS does the backend transition the workflow into placement mode.

Once the die is validated successfully, the backend records the result (`dieset_value`, `dieset_passed`, and `dieset_img`) against the hose end assembly, and updates the branch-level state (`last_validated_die`) so repeated validations are avoided when the same die remains installed for subsequent crimps. The frontend and backend remain connected through

the same WebSocket, with the backend explicitly communicating the current mode using `is_placement_flag`, enabling a seamless operator experience while maintaining a strict gate before placement evaluation.

6.5.2. Different Cases and Backend Logic

Engaged-pixel reference missing for the suggested die

This case occurs when the system cannot retrieve the required engaged-pixel reference for the recommended die size during Step 3 initialisation. The engaged-pixel values define the calibrated crimper width thresholds that determine when the crimper is fully closed or open. If this configuration entry is missing for the suggested die, the backend raises an exception before proceeding with die validation or placement evaluation.

When this condition is detected, the system immediately records Step 3 as failed for the current hose end, logs the issue for traceability, and sets the `engage_pixel_not_found_flag` in the response package. The workflow is then terminated for the entire Step 3 (`finish_flag = true`), allowing the frontend to handle the failure appropriately.

This behaviour ensures that the system does not attempt die validation or placement evaluation without valid calibration parameters, highlighting configuration errors that require administrative correction.

Die check already completed for this hose end

This case occurs when the die validation has already been performed for a specific hose end and the result has been stored in the database (`dieset_passed` is no longer NULL). A typical scenario is when the operator completes the die validation stage but exits Step 3 by returning to the scan page before finishing the placement check. If the operator subsequently scans the same hose end barcode again, the backend retrieves the existing hose end record and detects that die validation has already been completed for that assembly.

As a result, when Step 3 is re-entered, the system does not re-enter die verification mode. Instead, it transitions directly into placement mode. This behaviour preserves workflow continuity while preventing redundant die validation for the same hose end.

Die check not completed, but not required

This case occurs when die validation has not yet been performed for the current hose end (`dieset_passed` is NULL), but validation is determined to be unnecessary because the branch's `last_validated_die` matches the recommended die size for the current assembly. In this situation, the system assumes that the correct die remains installed on the crimper from a previously validated operation. The backend therefore does not enter die verification mode. Instead, it automatically records the die check for the current hose end as “not needed” (stored in `dieset_value`) and proceeds directly to placement mode.

Die check required

This case occurs when the backend determines that die validation must be performed before proceeding to placement mode. Die verification mode is activated whenever the system cannot confidently assume that the correct die is currently installed on the crimper.

The first scenario is the first hose end processed after login. Upon login, the branch-level `last_validated_die` is reset to `NULL`. Since there is no previously validated die state available, the system cannot assume die correctness. Therefore, the first hose end entering Step 3 will always require die validation.

The second scenario occurs when a previous die validation was explicitly skipped by the operator. In this case, the skip logic resets `last_validated_die` to `NULL`. This ensures that the system does not carry forward an unverified die state. As a result, the next hose end entering Step 3 will require a full die validation.

The third scenario occurs when `last_validated_die` is not `NULL`, but its value differs from the recommended die size (`suggested_die_size`) for the current assembly. This indicates that the die required for the current hose end is different from the previously validated die. Since the die set may have been changed, the system must re-enter die verification mode to confirm correctness before allowing placement to proceed.

Idle in die verification mode

This case occurs after the system has determined that die validation is required, but the operator has not yet taken any action. Once Step 3 begins and the backend enters die verification mode, it waits for explicit input from the frontend—either a validation trigger or a skip instruction.

During this idle state, the backend continues streaming frames from the crimper camera but does not execute the die validation inference pipeline. The response package sent to the frontend indicates that the system is in die mode (`is_placement_flag = false`) and that no final decision has been made (`die_passed_flag = "NULL"`). The workflow does not progress until the operator presses the “Validate” button or chooses to skip.

Operator triggers validation

This case occurs when the operator actively initiates die validation by pressing the “Validate” button on the frontend. The frontend sends a `die_validation_flag` signal to the backend through the Step 3 WebSocket, indicating that the system should execute the die verification logic.

Upon receiving this trigger, the backend runs the die validation inference pipeline. Unlike placement evaluation — which processes frames continuously and updates results one frame at a time — die validation is executed as a short, blocking evaluation window (approximately one second). During this interval, multiple frames are sampled and aggregated to produce a final stable verification result. The detected die size is compared against the recommended die size (`suggested_die_size`) for the current hose end. The evaluation result is mapped into a three-state outcome: `PASS`, `FAIL`, or `UNCERTAIN`, which is returned to the frontend via `die_passed_flag`.

Die validation PASS

This case occurs when the die evaluation result confirms that the installed die matches the recommended die size for the current hose end. After the operator triggers validation, the backend runs the inference pipeline and receives a `PASS` decision from the verification logic.

When a PASS outcome is obtained, the backend persists the die validation result to the database. Specifically, it records the detected die value (`dieset_value`), sets `dieset_passed` to `True`, and stores the captured validation image (`dieset_img`) for traceability. In addition, the branch-level `last_validated_die` is updated to the recommended die size. This allows subsequent hose ends using the same die to bypass redundant validation, provided the die remains unchanged.

Following successful persistence, the backend transitions from die verification mode into placement mode. The frontend is updated accordingly (`is_placement_flag = true`), and Step 3 proceeds with normal placement evaluation.

Die validation FAIL

This case occurs when the die evaluation determines that the installed die does not match the recommended die size for the current hose end. After the operator triggers validation, the backend runs the die verification logic and receives a FAIL outcome, indicating a clear mismatch between the detected die and the expected specification.

When a FAIL result is produced, the backend updates the frontend with `die_passed_flag = "FAIL"` but does not transition into placement mode. The system remains in die verification mode (`is_placement_flag = false`), preventing the workflow from progressing.

No branch-level `last_validated_die` update is performed in this case, since the die has not been successfully validated. The operator is expected to replace the die set with the correct one and trigger validation again. This behaviour ensures that incorrect tooling is intercepted before crimp engagement, preserving product quality and process integrity.

Die validation UNCERTAIN

This case occurs when the die evaluation cannot determine whether the installed die matches the recommended die size. After the operator triggers validation, the backend runs the die verification pipeline, but the similarity score does not meet the criteria required for a definitive PASS or FAIL decision. As a result, the outcome is mapped to UNCERTAIN.

An UNCERTAIN result typically arises from conditions such as partial occlusion of markers, motion blur, inconsistent lighting, or oil/dirt contamination. In this situation, the backend returns `die_passed_flag = "UNCERTAIN"` to the frontend and remains in die verification mode (`is_placement_flag = false`). The workflow does not proceed to placement.

No database update is committed for a successful validation, and `last_validated_die` remains unchanged. The operator may adjust the camera view, improve visibility, or physically inspect the die before triggering validation again.

Operator skips die validation

This case occurs when the operator chooses to bypass die validation by triggering the `die_skip` action from the frontend. This may happen if the operator has manually verified the die or needs to proceed despite system uncertainty.

When a skip instruction is received, the backend records the die check for the current hose end as skipped. The system updates the assembly record by storing an appropriate value in

`dieset_value` (“Skipped”) and setting `dieset_passed` to `False`, ensuring that the skip event is traceable. Any provided skip reason is logged for audit purposes.

Importantly, the branch-level `last_validated_die` is reset to `NULL`. This prevents the system from assuming that the die is validated for the current hose ends. As a result, the next hose end entering Step 3 will require a full die validation. After recording the skip event, the backend transitions into placement mode, allowing the workflow to proceed while clearly marking that die validation was not completed.

6.5.3. Skip Handling Change

Previously, the skip flow in Step 3 relied on a disconnect-driven mechanism. When the operator pressed Skip, the frontend first sent an error payload, which caused the backend loop to stop progressing and enter an effective idle state. After the operator entered a reason and confirmed, the frontend sent the final skip reason and then disconnected the WebSocket. The backend would then interpret the disconnect inside the `WebSocketDisconnect` handler and apply the skip logic there. While functional, this design was unintuitive because the decision to skip was finalised and recorded only after a disconnect event, rather than as a deliberate workflow command.

The updated design moves skip handling into the standard request/response flow. After the operator presses Skip, the frontend temporarily stops sending a valid email (the `email` field becomes empty), which keeps Step 3 in a safe idle state and prevents it from progressing. Once the operator confirms a skip reason, the frontend sends the email plus the skip reason using either `die_skip` or `place_skip`. The backend then immediately executes the skip logic: it marks the relevant check as failed, logs the reason, and returns a check finish response to the frontend (`die_passed_flag` for die skip or `finish_flag` for placement skip). After sending this final package, the backend closes the WebSocket itself.

This change makes the workflow more deterministic and semantically correct: skips are now handled as explicit commands with a clear acknowledgement from the backend, rather than being inferred indirectly through a network disconnect.

Thanks for taking the time to read through this!
